



BITS Pilani
Pilani Campus

Unsupervised Deep Learning ZG513

Dr. Sugata Ghosal
sugata.ghosal@pilani.bits-pilani.ac.in



Session 3

These slides are prepared by the instructor, with grateful acknowledgement of many others who made their course materials freely available online.

Agenda



- SUPERVISED VS UNSUPERVISED LEARNING EXPERIENCE
- AUTOENCODERS
 - UNDERCOMPLETE AE
 - REGULARIZED AE
 - CONVOLUTIONAL AE
 - DENOISING AE
 - SPARSE AE
- DEEP AUTOENCODERS

SUPERVISED LEARNING

DATA: m training examples (X, y)
 X is the data and y is the label.

GOAL: Learn a function to map data to the label. $y = f(X)$

EXAMPLE TASKS: Classification, regression, object detection, semantic segmentation, image captioning, etc.

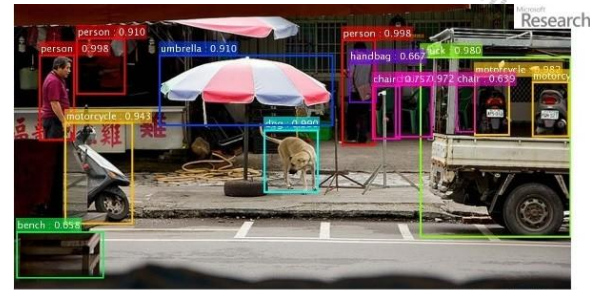


FIGURE: Object recognition: He: Faster R-CNN



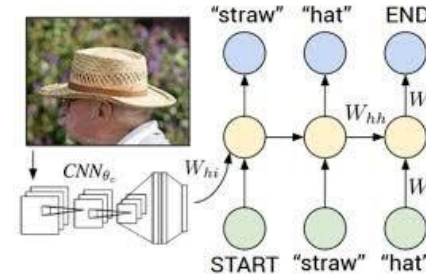
FIGURE: OCR: Hui Li, Wang, & Shen

SUPERVISED LEARNING

DATA: m training examples (X, y)
 X is the data and y is the label.

GOAL: Learn a function to map data to the label. $y = f(X)$

EXAMPLE TASKS: Classification, regression, object detection, semantic segmentation, image captioning, etc.



Andrej Karpathy, Fei-Fei Li

FIGURE: Image Captioning:
Andrej Karpathy, Fei-Fei Li



FIGURE: Semantic Segmentation

UNSUPERVISED LEARNING

DATA: m training examples (X)

X is the data. No Labels !!!

GOAL: Learn some underlying hidden structure of the data.

EXAMPLE TASKS: Clustering, dimensionality reduction, feature learning, density estimation, etc.

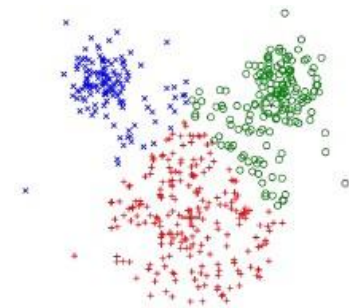


FIGURE: k-means Clustering

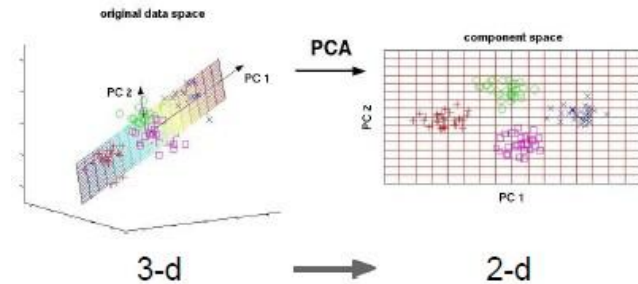


FIGURE: Dimensionality Reduction

UNSUPERVISED LEARNING

DATA: m training examples (X)
 X is the data. No Labels !!!

GOAL: Learn some underlying hidden structure of the data.

EXAMPLE TASKS: Clustering, dimensionality reduction, feature learning, density estimation, etc.

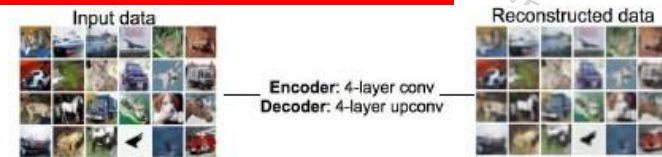
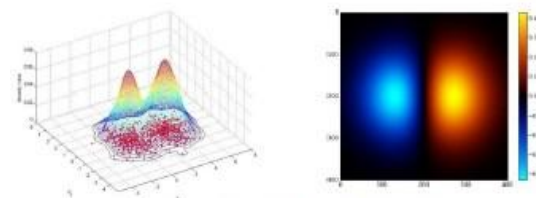


FIGURE: Feature Learning using Autoencoders



1-d density estimation



2-d density estimation

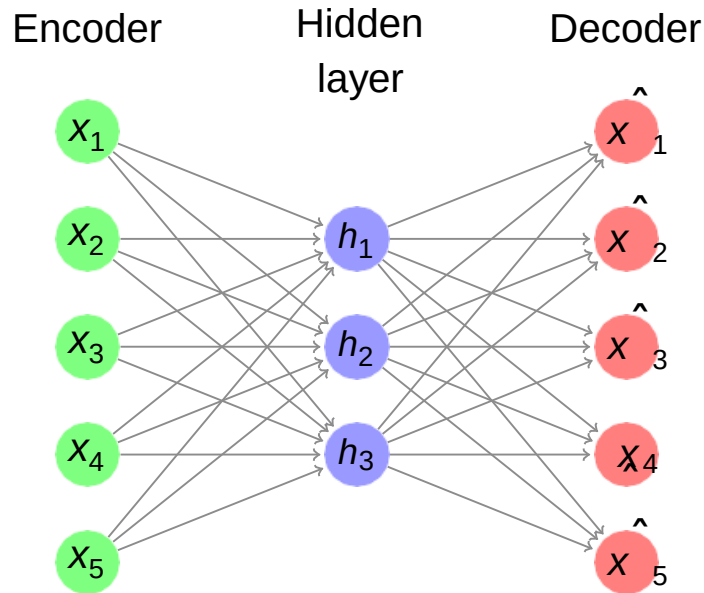
FIGURE: Density Estimation

AUTOENCODERS



- **Unsupervised** experience.
- Learns a lower-dimensional feature representation called **code**; from unlabeled training data.
- The code can be used as features also. Autoencoders are efficient **feature detectors**.
- Autoencoders perform **dimensionality reduction**.
- Trained to attempt to copy its input to its output.

AUTOENCODER ARCHITECTURE

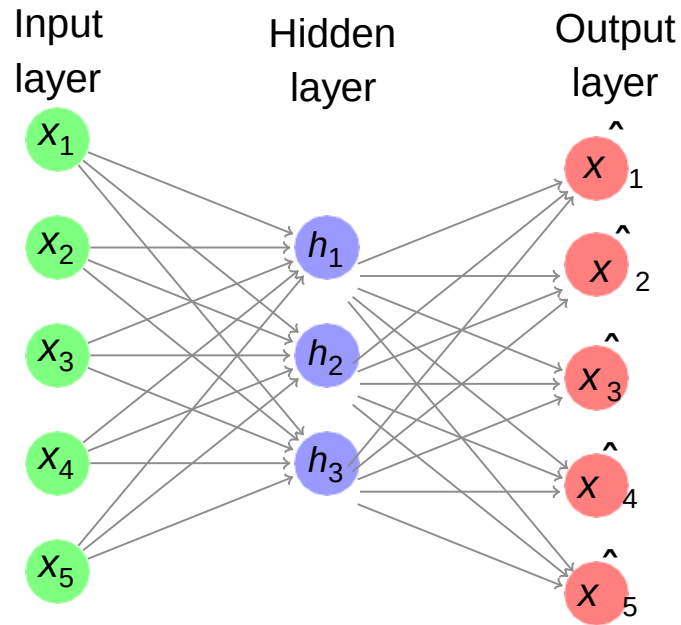


$$\hat{h} = f(W_e x_i + b)$$

$$\hat{x}_i = g(W_d h + c)$$

- An autoencoder is a special type of feed forward neural network which does the following.
- **Encodes** its input x_i into a hidden representation h .
- **Decodes** the input again from this hidden representation.
- The model is trained to minimize a certain loss function which will ensure that $\hat{x}_i$ is close to x_i .

AUTOENCODER ARCHITECTURE



$$h = f(W_e x_i + b)$$

$$\hat{x}_i = g(W_d h + c)$$

- Feedforward deep neural network
- Contains input layer, hidden layer and output layers.
- The number of output neurons is exactly same as the number of input neurons
- The hidden layer describes the code used to represent the input.
- All techniques and optimizations of DNN are applicable.
- Train the autoencoder such that features can be used to reconstruct original data.

UNDERCOMPLETE AUTOENCODERS

innovate

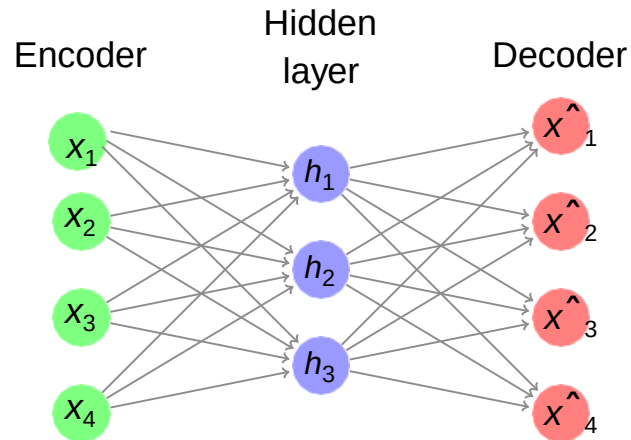
achieve

lead

- One hidden layer which has lesser units than input layer.

$$\dim(h) < \dim(x)$$

- Input and output layers have equal number of neurons.
- h is a loss-free encoding of x_i . It captures all important characteristics of x_i .



UNDERCOMPLETE AUTOENCODERS

■ Components of Autoencoders

- Encoder (recognition network)

- ✓ converts input to code.

$$h = f(x) = f(W_e x + b)$$

- ✓ Code h has a smaller representation than input x .

- Decoder (generative network)

- ✓ produces a reconstruction.

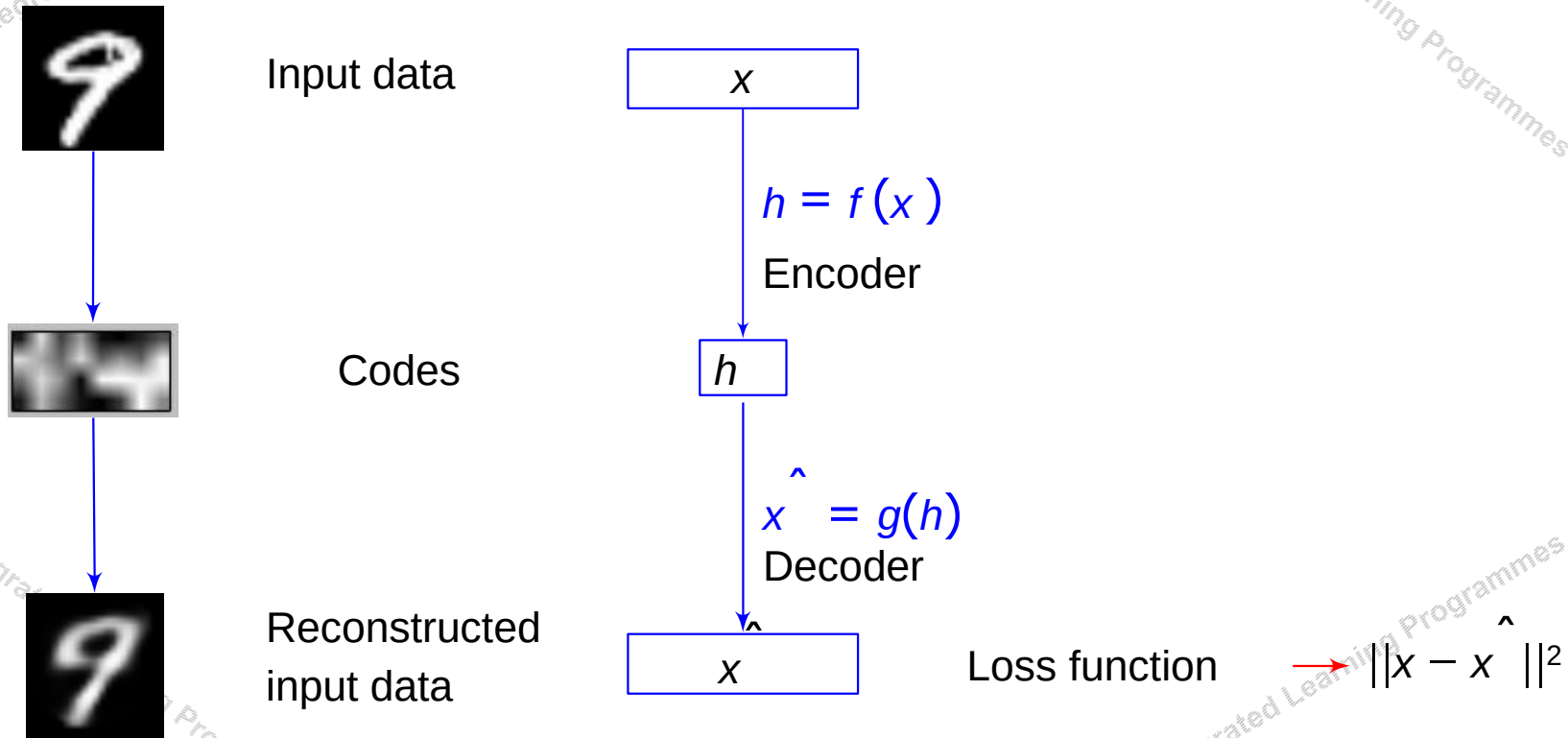
$$\hat{x} = g(h) = g(W_d h + c) = g(f(x))$$

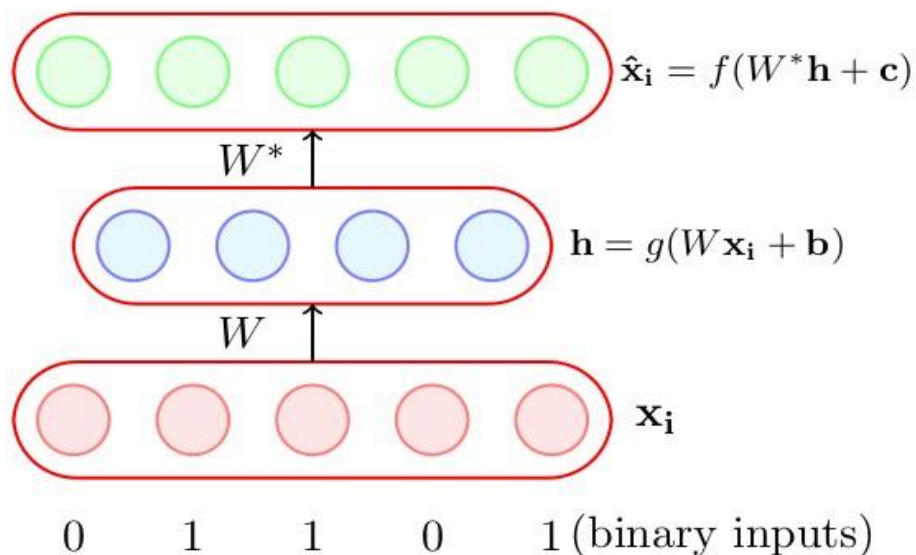
- Loss function

- ✓ reconstruction loss which penalizes when outputs are different from input.
- ✓ Mean squared error

$$L(x, \hat{x}) = ||x - \hat{x}||^2$$

UNDERCOMPLETE AUTOENCODERS





- Suppose all our inputs are binary (each $x_{ij} \in \{0, 1\}$)
- Which of the following functions would be most apt for the decoder?

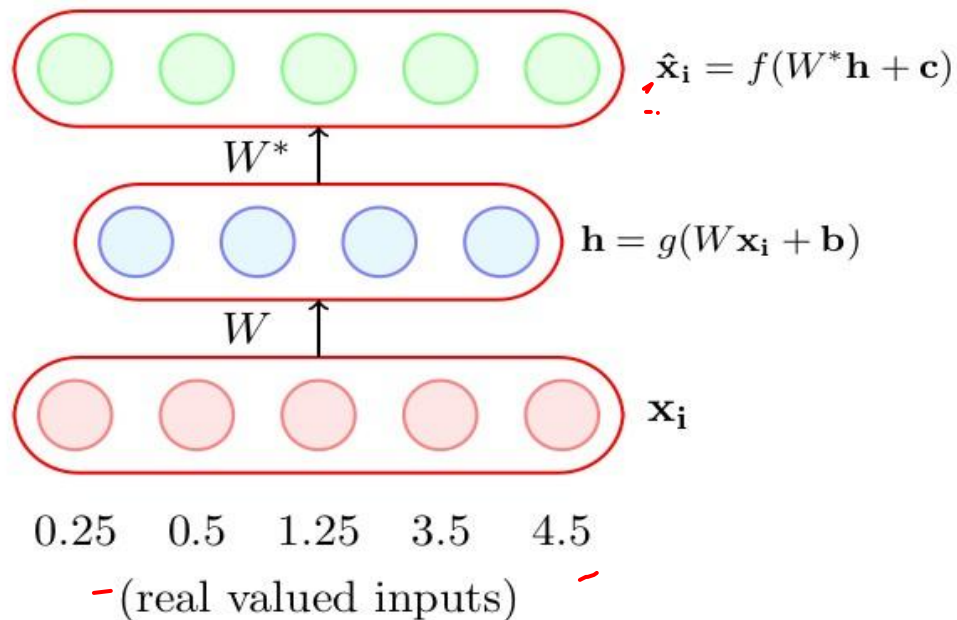
$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = W^*\mathbf{h} + \mathbf{c}$$

$$\hat{\mathbf{x}}_i = \text{logistic}(W^*\mathbf{h} + \mathbf{c})$$

- Logistic as it naturally restricts the outputs to be between 0 and 1

g is typically chosen as the sigmoid function



Again, g is typically chosen as the sigmoid function

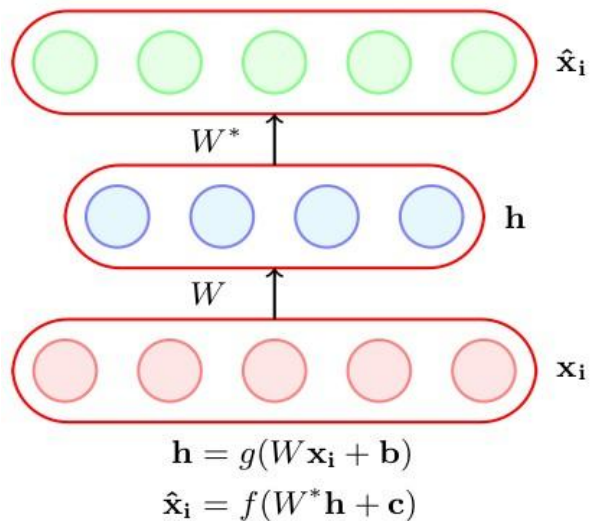
- Suppose all our inputs are real (each $x_{ij} \in \mathbb{R}$)
- Which of the following functions would be most apt for the decoder?

$$\hat{\mathbf{x}}_i = \tanh(W^*\mathbf{h} + \mathbf{c})$$

$$\hat{\mathbf{x}}_i = W^*\mathbf{h} + \mathbf{c}$$

$$\hat{\mathbf{x}}_i = \text{logistic}(W^*\mathbf{h} + \mathbf{c})$$

- What will logistic and tanh do?
- They will restrict the reconstructed $\hat{\mathbf{x}}_i$ to lie between $[0,1]$ or $[-1,1]$, whereas we want $\hat{\mathbf{x}}_i \in \mathbb{R}^n$

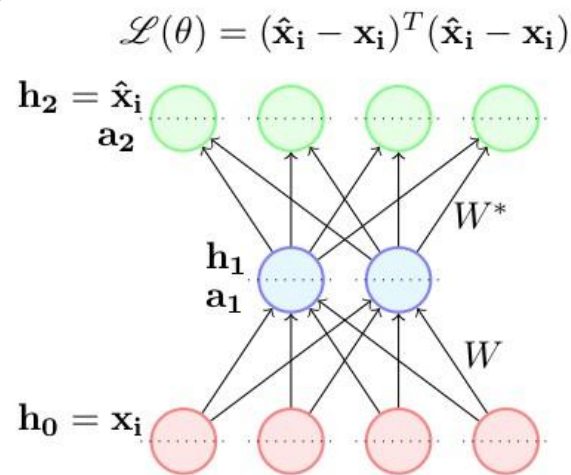


- Consider the case when the inputs are real valued
- The objective of the autoencoder is to reconstruct $\hat{\mathbf{x}}_i$ to be as close to $\mathbf{x}_i$ as possible
- This can be formalized using the following objective function:

$$\min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

$$i.e., \min_{W, W^*, \mathbf{c}, \mathbf{b}} \frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

- We can then train the autoencoder just like a regular feedforward network using back-propagation
- All we need is a formula for $\frac{\partial \mathcal{L}(\theta)}{\partial W^*}$ and $\frac{\partial \mathcal{L}(\theta)}{\partial W}$ which we will see now



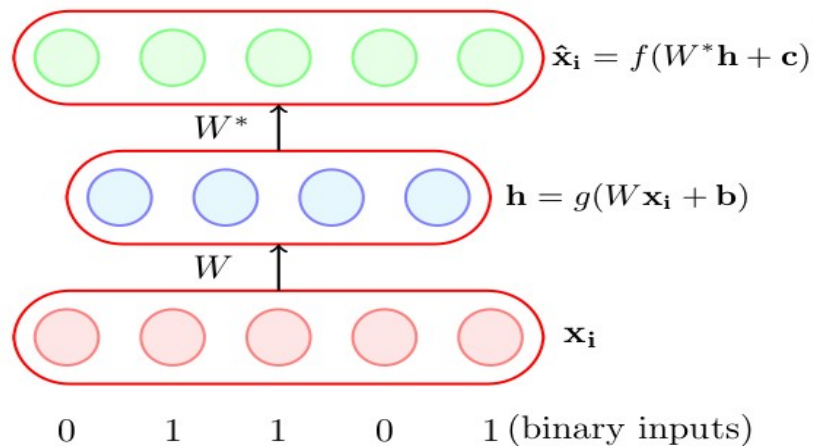
- Note that the loss function is shown for only one training example.

- $$\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial W^*}}$$

- $$\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \boxed{\frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$$

- We have already seen how to calculate the expression in the boxes when we learnt backpropagation

$$\begin{aligned} \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} &= \frac{\partial \mathcal{L}(\theta)}{\partial \hat{\mathbf{x}}_i} \\ &= \nabla_{\hat{\mathbf{x}}_i} \{ (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i) \} \\ &= 2(\hat{\mathbf{x}}_i - \mathbf{x}_i) \end{aligned}$$



What value of $\hat{x}_{ij}$ will minimize this function?

- If $x_{ij} = 1$?
- If $x_{ij} = 0$?

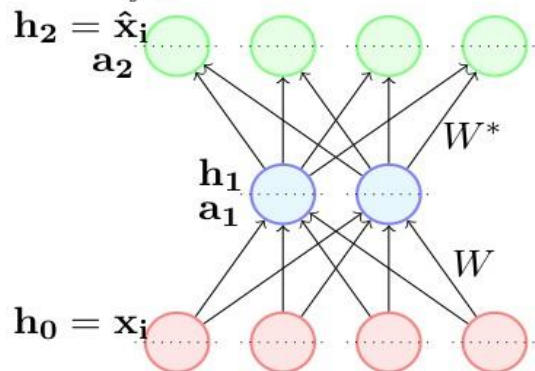
Indeed the above function will be minimized when $\hat{x}_{ij} = x_{ij}$!

- Consider the case when the inputs are binary
- We use a sigmoid decoder which will produce outputs between 0 and 1, and can be interpreted as probabilities.
- For a single n -dimensional i^{th} input we can use the following loss function

$$\min \left\{ - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij})) \right\}$$

- Again we need is a formula for $\frac{\partial \mathcal{L}(\theta)}{\partial W^*}$ and $\frac{\partial \mathcal{L}(\theta)}{\partial W}$ to use backpropagation

$$\mathcal{L}(\theta) = - \sum_{j=1}^n (x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}))$$



$$\frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} = \begin{pmatrix} \frac{\partial \mathcal{L}(\theta)}{\partial h_{21}} \\ \frac{\partial \mathcal{L}(\theta)}{\partial h_{22}} \\ \vdots \\ \frac{\partial \mathcal{L}(\theta)}{\partial h_{2n}} \end{pmatrix}$$

- $\frac{\partial \mathcal{L}(\theta)}{\partial W^*} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial W^*}}$
- $\frac{\partial \mathcal{L}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \boxed{\frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial W}}$

- We have already seen how to calculate the expressions in the square boxes when we learnt BP
- The first two terms on RHS can be computed as:

$$\frac{\partial \mathcal{L}(\theta)}{\partial h_{2j}} = -\frac{x_{ij}}{\hat{x}_{ij}} + \frac{1 - x_{ij}}{1 - \hat{x}_{ij}}$$

$$\frac{\partial h_{2j}}{\partial a_{2j}} = \sigma(a_{2j})(1 - \sigma(a_{2j}))$$

AUTOENCODERS AND PCA

innovate

achieve

lead

- The encoder of a linear autoencoder is equivalent to PCA if we
 - use a linear encoder
 - use a linear decoder
 - use a squared error loss function
 - normalize the inputs to

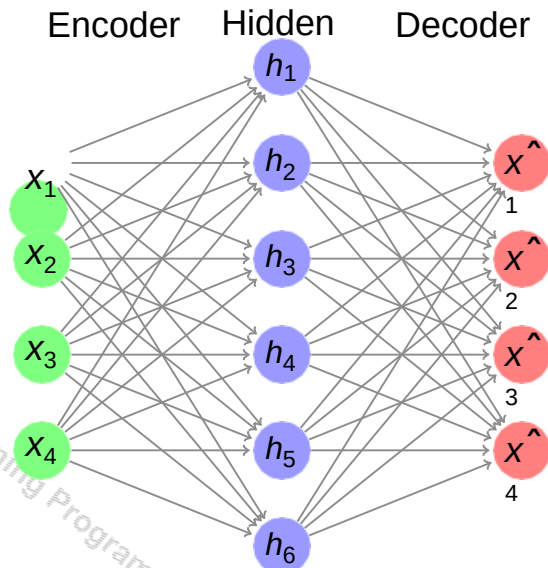
$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \frac{1}{m} \sum_{k=1}^m x_{kj} \right)$$

OVERCOMPLETE AUTOENCODERS

- One hidden layer which has more units than input layer.

$$\dim(h) \geq \dim(x)$$

- Learn a trivial encoding by simply copying x_i into h and then copying h into $\hat{x}_i$.

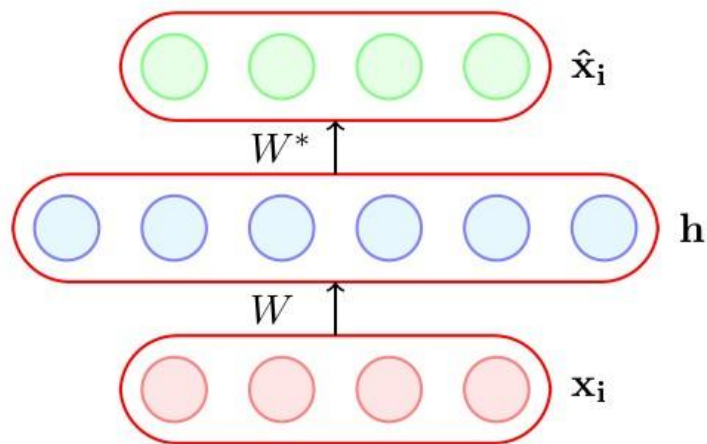


$$h = g(W_e x_i + b)$$
$$\hat{x}_i = f(W_d h + c)$$

REGULARIZED AUTOENCODERS



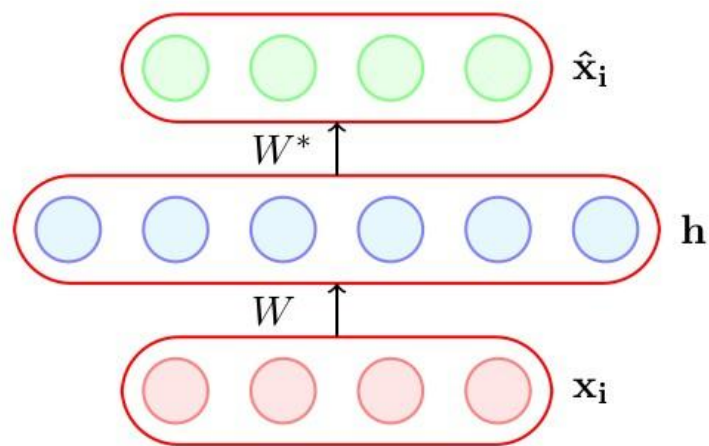
- Autoencoders fail to learn anything useful if the
 - › the encoder and decoder are given too much capacity.
 - › the hidden code is allowed to have dimension equal to the input.
 - › the hidden code has dimension greater than the input. (overcomplete case)
- Use Regularized autoencoders to resolve the above problems.
- To avoid poor generalization, introduce regularization.
- Introduces properties
 - › sparsity of the representation
 - › smallness of the derivative of the representation
 - › robustness to noise or to missing inputs.



- The simplest solution is to add a L_2 -regularization term to the objective function

$$\min_{\theta, w, w^*, \mathbf{b}, \mathbf{c}} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2 + \lambda \|\theta\|^2$$

- This is very easy to implement and just adds a term λW to the gradient $\frac{\partial \mathcal{L}(\theta)}{\partial W}$ (and similarly for other parameters)



- Another trick is to tie the weights of the encoder and decoder i.e., $W^* = W^T$
- This effectively reduces the capacity of Autoencoder and acts as a regularizer

CONVOLUTIONAL AUTOENCODERS



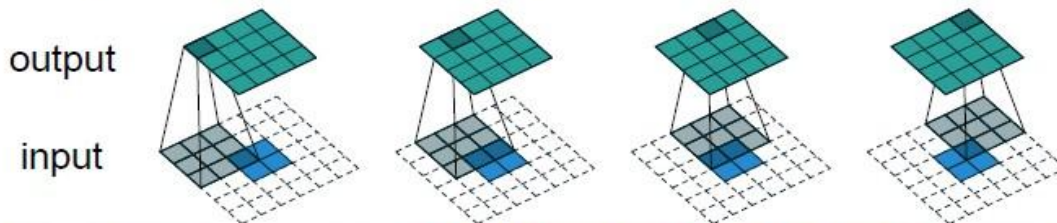
- Allows us to increase the size of the output feature map compared to the input feature map.
- Fractionally strided convolution is another term.
- **Strides** In transposed convolutions, we stride over the output; hence, larger strides will result in larger outputs (opposite to regular convolutions).

Regular Convolution:



Figure 2.1: (No padding, unit strides) Convoluting a 3×3 kernel over a 4×4 input using unit strides (i.e., $i = 4$, $k = 3$, $s = 1$ and $p = 0$).

Transposed Convolution (emulated with direct convolution):



Dumoulin, Vincent, and Francesco Visin. "[A guide to convolution arithmetic for deep learning](#)." *arXiv preprint arXiv:1603.07285* (2016).

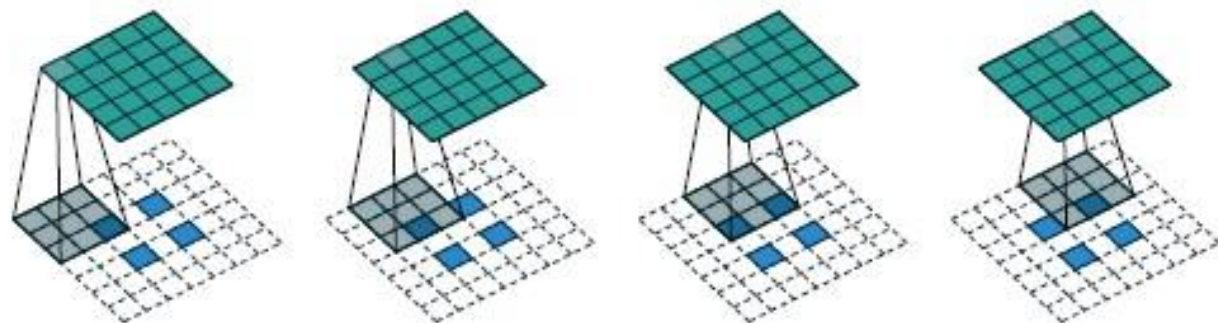


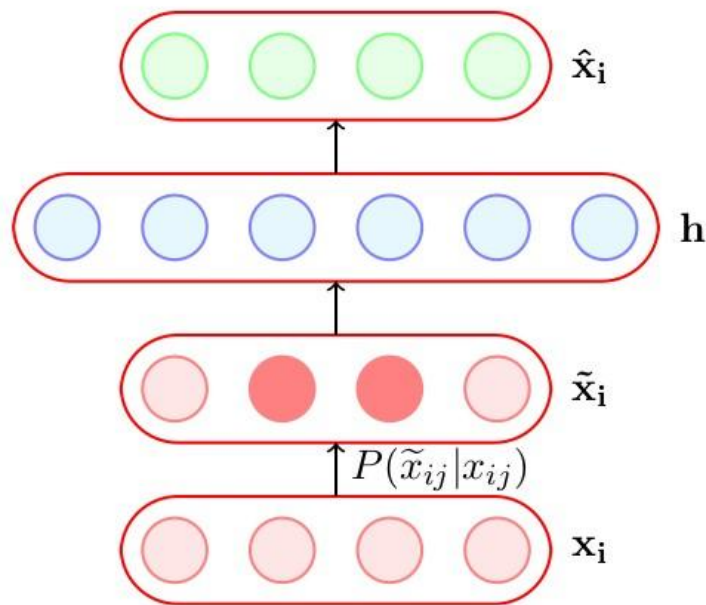
Figure 4.5: The transpose of convolving a 3×3 kernel over a 5×5 input using 2×2 strides (i.e., $i = 5, k = 3, s = 2$ and $p = 0$). It is equivalent to convolving a 3×3 kernel over a 2×2 input (with 1 zero inserted between inputs) padded with a 2×2 border of zeros using unit strides

Dumoulin, Vincent, and Francesco Visin. "[A guide to convolution arithmetic for deep learning](#)." *arXiv preprint arXiv:1603.07285* (2016).

DENOISING AUTOENCODERS (DAE)



- Receives a corrupted data point as input and is trained to predict the original, uncorrupted data point as its output.
- $x_{\sim}$ is a copy of x corrupted by some form of noise.
- The corruption process adds some noise to x according to the conditional distribution $C(x_{\sim}, x)$.
- Denoising autoencoders removes this corruption.

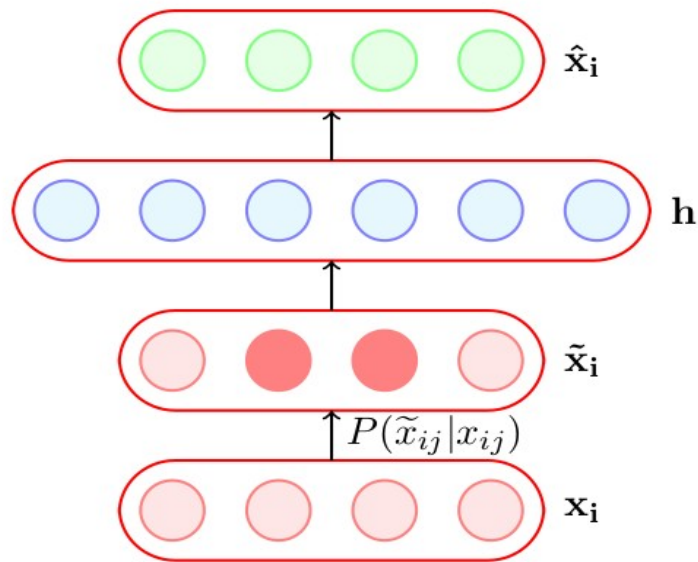


- A denoising encoder simply corrupts the input data using a probabilistic process ($P(\tilde{x}_{ij}|x_{ij})$) before feeding it to the network
- A simple $P(\tilde{x}_{ij}|x_{ij})$ used in practice is the following

$$P(\tilde{x}_{ij} = 0|x_{ij}) = q$$

$$P(\tilde{x}_{ij} = x_{ij}|x_{ij}) = 1 - q$$

- In other words, with probability q the input is flipped to 0 and with probability $(1 - q)$ it is retained as it is



For example, it will have to learn to reconstruct a corrupted x_{ij} correctly by relying on its interactions with other elements of $\mathbf{x}_i$

- How does this help ?
- This helps because the objective is still to reconstruct the original (un-corrupted) $\mathbf{x}_i$

$$\arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$$

- It no longer makes sense for the model to copy the corrupted $\tilde{\mathbf{x}}_i$ into $h(\tilde{\mathbf{x}}_i)$ and then into $\hat{\mathbf{x}}_i$ (the objective function will not be minimized by doing so)
- Instead the model will now have to capture the characteristics of the data correctly.

Task: Hand-written digit recognition



Figure: MNIST Data

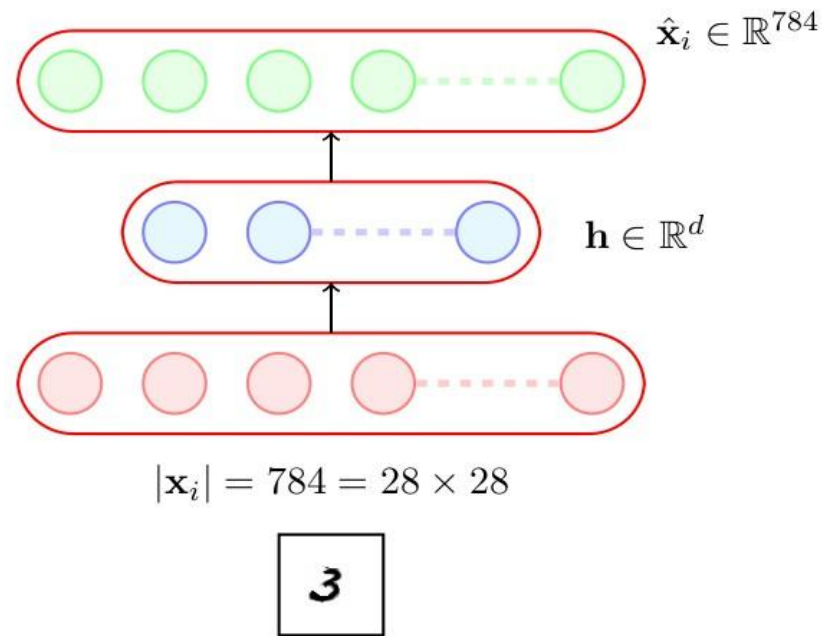


Figure: AE approach (first learn important characteristics of data)

Task: Hand-written digit recognition

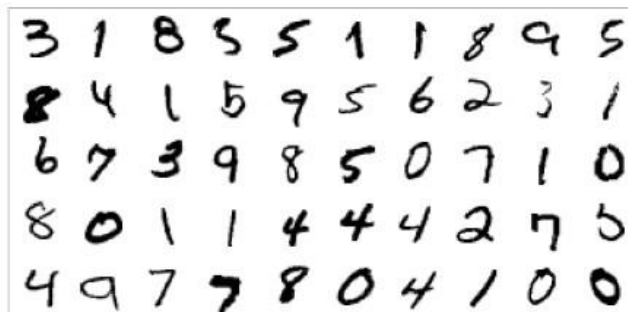


Figure: MNIST Data

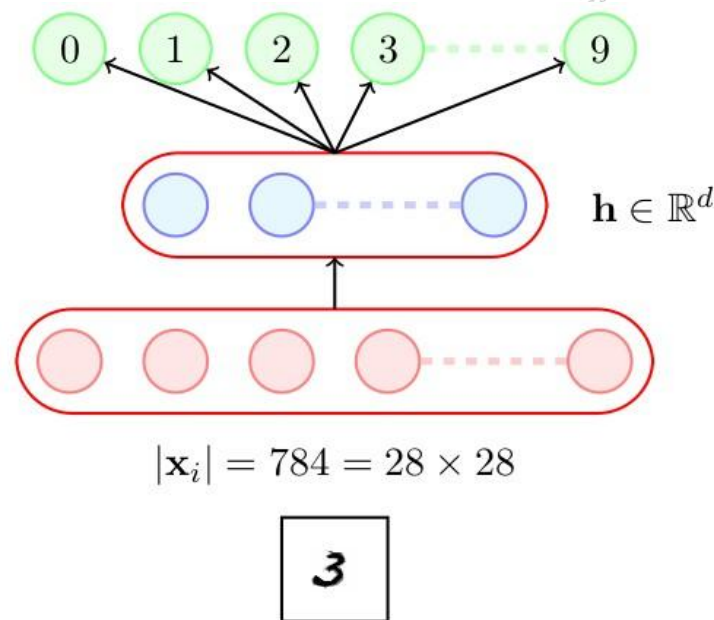
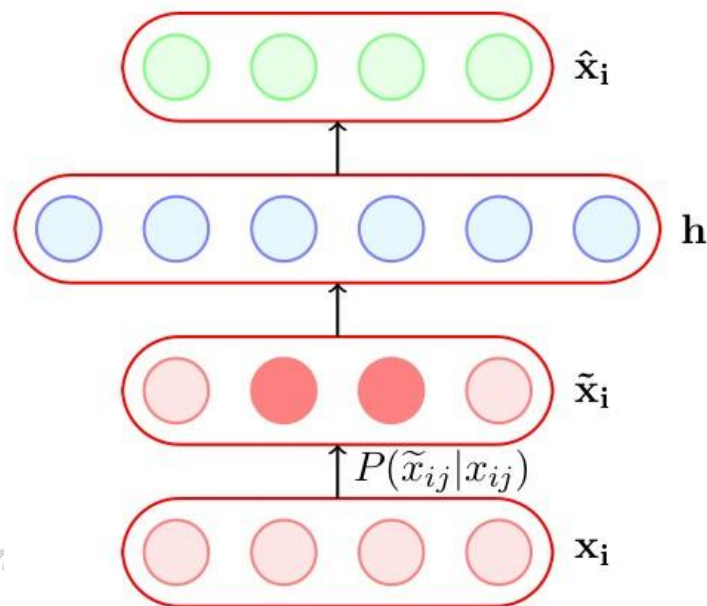


Figure: AE approach (and then train a classifier on top of this hidden representation)



- We saw one form of $P(\tilde{x}_{ij}|x_{ij})$ which flips a fraction q of the inputs to zero
- Another way of corrupting the inputs is to add a Gaussian noise to the input

$$\tilde{x}_{ij} = x_{ij} + \mathcal{N}(0, 1)$$

- We will now use such a denoising AE on a different dataset and see their performance

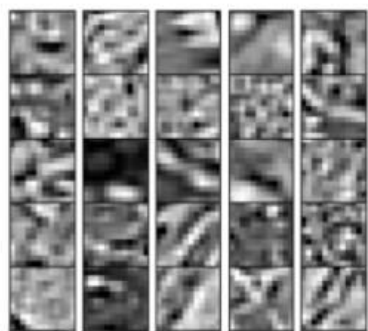


Figure: Data

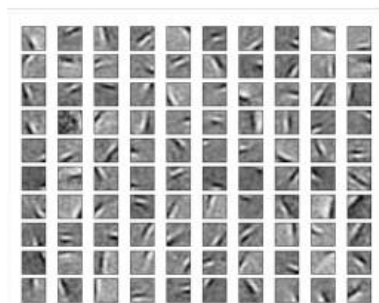


Figure: AE filters

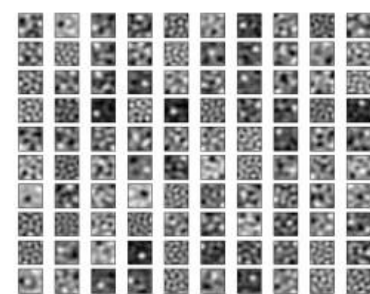
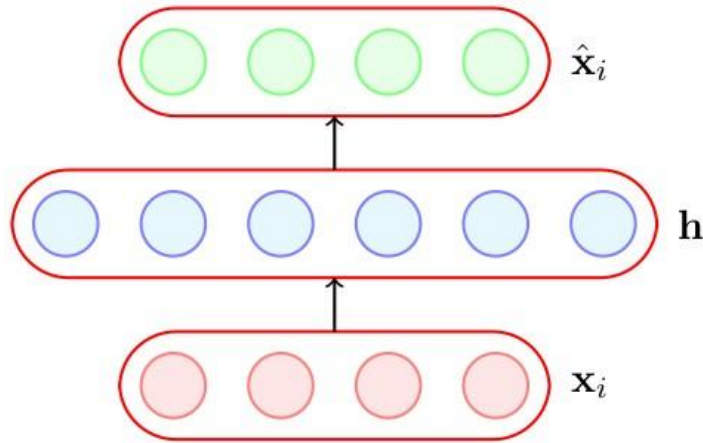


Figure: Weight decay filters

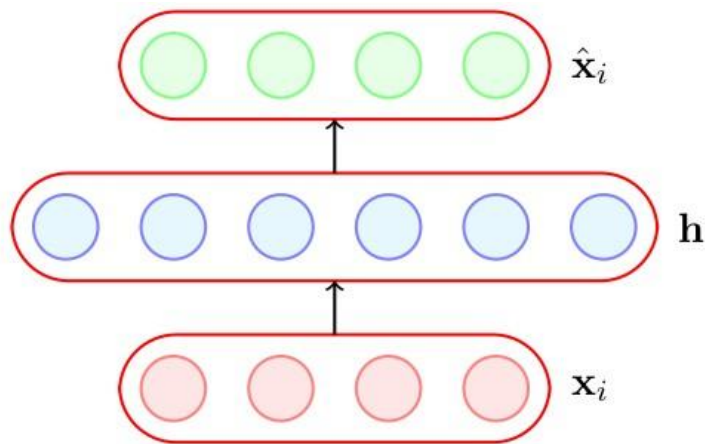
- The hidden neurons essentially behave like edge detectors
- PCA does not give such edge detectors

Sparse AE



- A hidden neuron with sigmoid activation will have values between 0 and 1
- We say that the neuron is activated when its output is close to 1 and not activated when its output is close to 0.
- A sparse autoencoder tries to ensure the neuron is inactive most of the times.

Sparse AE



The average value of the activation of a neuron l is given by

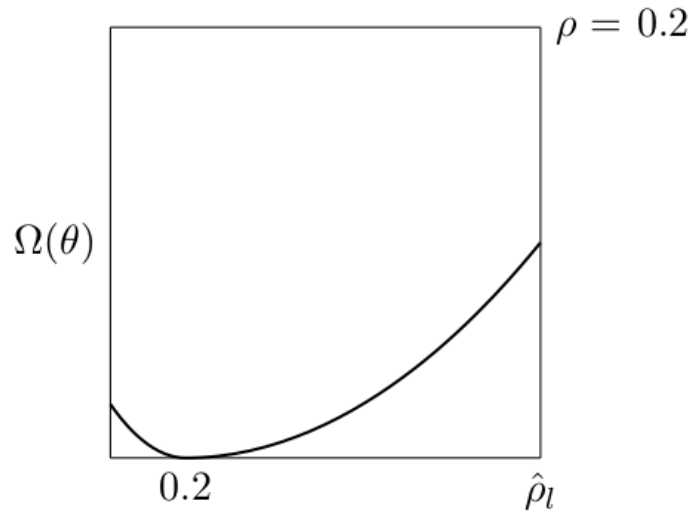
$$\hat{\rho}_l = \frac{1}{m} \sum_{i=1}^m h(\mathbf{x}_i)_l$$

- If the neuron l is sparse (i.e. mostly inactive) then $\hat{\rho}_l \rightarrow 0$
- A sparse autoencoder uses a sparsity parameter ρ (typically very close to 0, say, 0.005) and tries to enforce the constraint $\hat{\rho}_l = \rho$
- One way of ensuring this is to add the following term to the objective function

$$\Omega(\theta) = \sum_{l=1}^k \rho \log \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_l}$$

- When will this term reach its minimum value and what is the minimum value? Let us plot it and check.

Loss Function



- The function will reach its minimum value(s) when $\hat{\rho}_l = \rho$.

$$\Omega(\theta) = \sum_{l=1}^k \rho \log \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_l}$$

Can be re-written as

$$\Omega(\theta) = \sum_{l=1}^k \rho \log \rho - \rho \log \hat{\rho}_l + (1 - \rho) \log (1 - \rho) - (1 - \rho) \log (1 - \hat{\rho}_l)$$

By Chain rule:

$$\frac{\partial \Omega(\theta)}{\partial W} = \frac{\partial \Omega(\theta)}{\partial \hat{\rho}} \cdot \frac{\partial \hat{\rho}}{\partial W}$$

$$\frac{\partial \Omega(\theta)}{\partial \hat{\rho}} = \left[\frac{\partial \Omega(\theta)}{\partial \hat{\rho}_1}, \frac{\partial \Omega(\theta)}{\partial \hat{\rho}_2}, \dots, \frac{\partial \Omega(\theta)}{\partial \hat{\rho}_k} \right]^T$$

For each neuron $l \in 1 \dots k$ in hidden layer, we have

$$\frac{\partial \Omega(\theta)}{\partial \hat{\rho}_l} = -\frac{\rho}{\hat{\rho}_l} + \frac{(1 - \rho)}{1 - \hat{\rho}_l}$$

and $\frac{\partial \hat{\rho}_l}{\partial W} = \mathbf{x}_i (g'(W^T \mathbf{x}_i + \mathbf{b}))^T$ (see next slide)

- Now,

$$\hat{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \Omega(\theta)$$

- $\mathcal{L}(\theta)$ is the squared error loss or cross entropy loss and $\Omega(\theta)$ is the sparsity constraint.
- We already know how to calculate $\frac{\partial \mathcal{L}(\theta)}{\partial W}$
- Let us see how to calculate $\frac{\partial \Omega(\theta)}{\partial W}$.
- Finally,

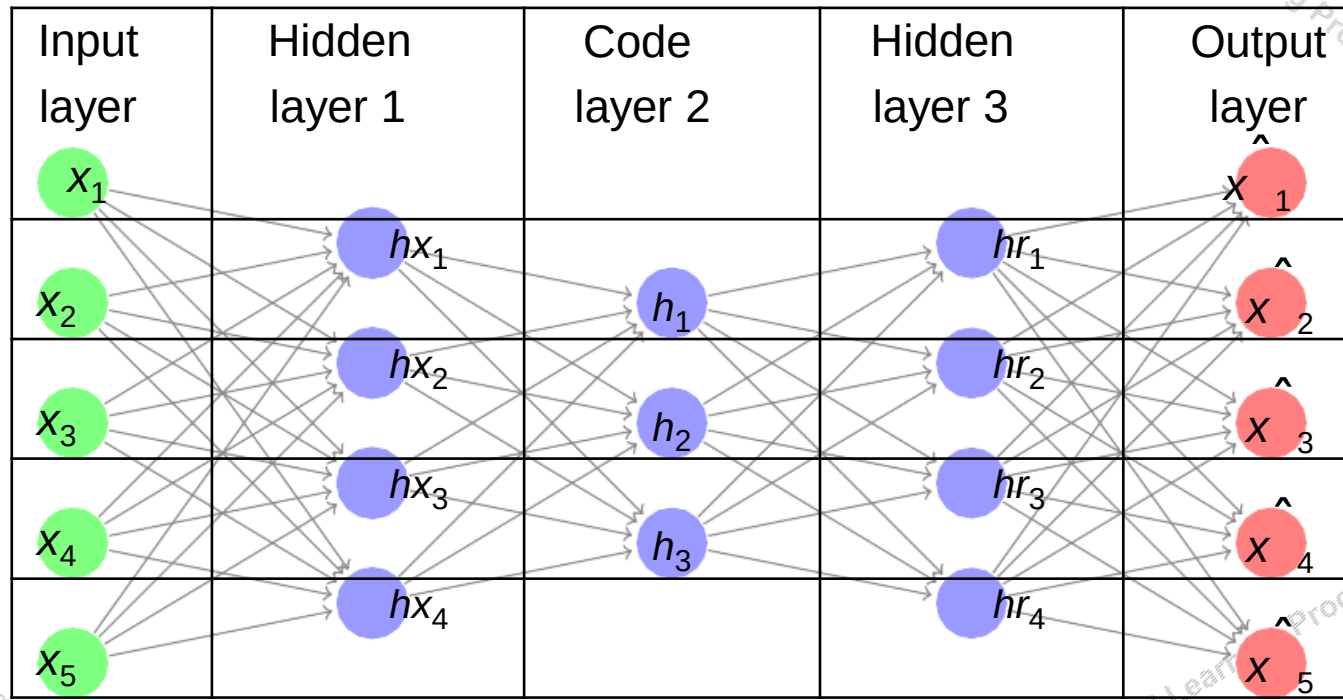
$$\frac{\partial \hat{\mathcal{L}}(\theta)}{\partial W} = \frac{\partial \mathcal{L}(\theta)}{\partial W} + \frac{\partial \Omega(\theta)}{\partial W}$$

(and we know how to calculate both terms on R.H.S)

DEEP AUTOENCODERS



- Input and output layers have hidden units.
- Code layer has lesser units than input layer.
- Depth can exponentially reduce the computational cost of representing some functions.
- Depth can exponentially decrease the amount of training data needed to learn some functions.
- Deep autoencoders yield much better compression than corresponding shallow or linear autoencoders .



AUTOENCODER APPLICATIONS



- Dimensionality reduction (representation learning)
- Information retrieval/semantic hashing tasks - we can store all database entries in a hash table that maps binary code vectors to entries
- Classification
- Denoising autoencoders
- Useful for segmentation and deep-feature
- Neural inpainting

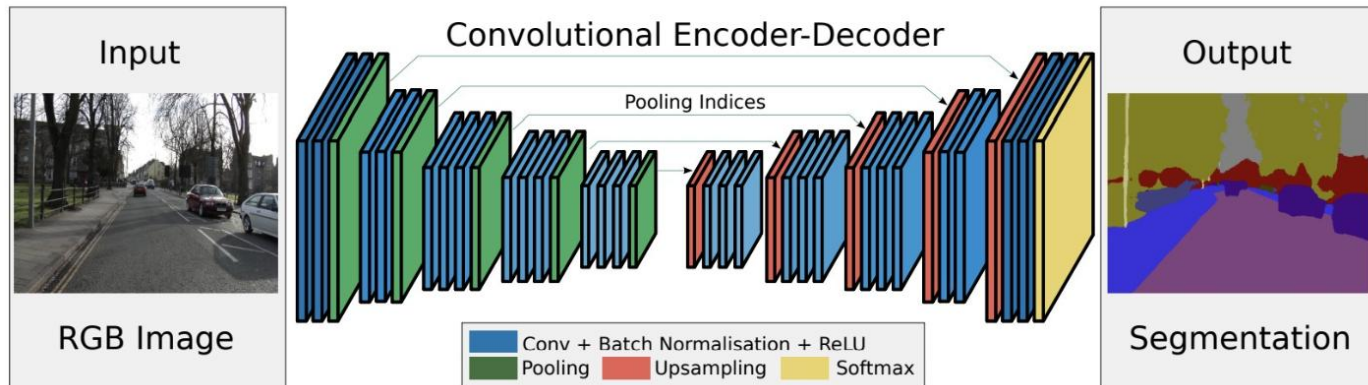
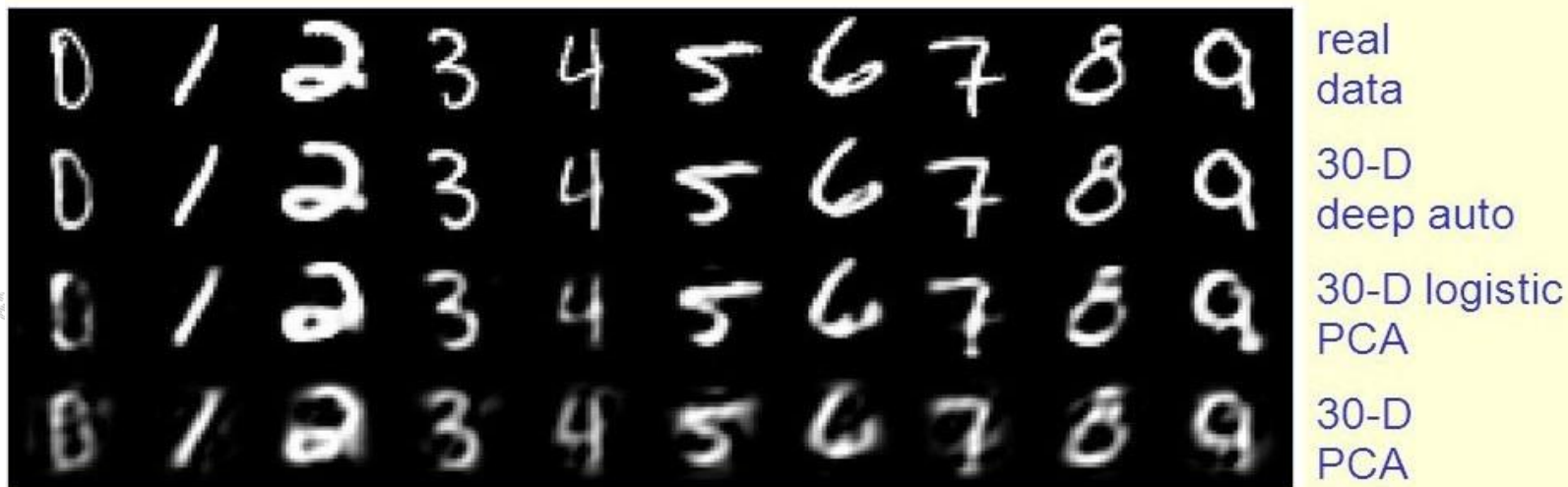


Fig. 2. An illustration of the SegNet architecture. There are no fully connected layers and hence it is only convolutional. A decoder upsamples its input using the transferred pool indices from its encoder to produce a sparse feature map(s). It then performs convolution with a trainable filter bank to densify the feature map. The final decoder output feature maps are fed to a soft-max classifier for pixel-wise classification.

A comparison of methods for compressing digit images to 30 real numbers.



AUTOENCODER – SUMMARY



- An autoencoder network is actually a pair of two connected networks, an encoder and a decoder.
- An encoder network takes in an input, and converts it into a smaller, dense representation.
- A decoder network can use to convert the dense representation back to the original input.



References

- 1 Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville
<https://www.deeplearningbook.org/>
- 2 Deep Learning with Python by Francois Chollet.
<https://livebook.manning.com/book/deep-learning-with-python/>